

Universidad Torcuato Di Tella

Métodos Analíticos y Programación

Prof. Hernán Czemerinski

Grafos: Relaciones, Conexiones y Ponderaciones

¿Por qué grafos?

Los grafos son estructuras matemáticas muy adecuadas para modelar sistemas del mundo real:

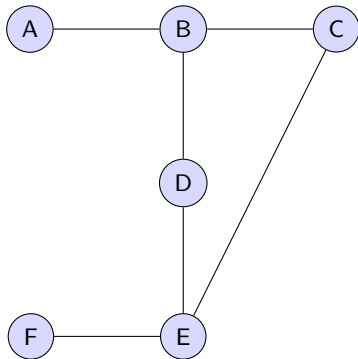
- Redes de transporte (subte, rutas, vuelos)
- Redes sociales (amistades, seguidores, menciones)
- Redes de comunicación o energía
- Recomendaciones o vínculos entre productos, usuarios o jugadores

Aunque parecen problemas distintos, todos comparten una estructura común:
entidades conectadas entre sí.

Ejemplo 1: Red de transporte urbano

Ejemplo: red simple de seis estaciones.

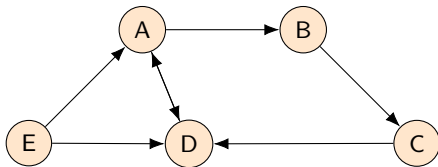
- Cada estación es un **nodo**.
- Cada conexión entre estaciones es una **arista**.



Ejemplo 2: Red social

Ejemplo: quién sigue a quién en una red social.

- Cada usuario es un **nodo**.
- Una **arista** indica que un usuario sigue a otro.



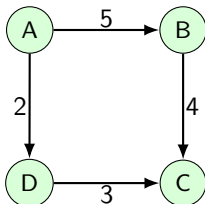
En una red social, las conexiones pueden ser:

- **Simétricas:** amistad (grafos no dirigidos)
- **Asimétricas:** seguidores o menciones (grafos dirigidos)

Ejemplo 3: Red de energía o flujo

Algunas redes representan **flujos** (de energía, datos, agua, etc.). Las aristas tienen un **peso** que indica capacidad o cantidad transportada.

Ejemplo: flujo de energía entre estaciones.



Los grafos como herramienta unificadora

Los grafos permiten representar y analizar relaciones complejas entre entidades:

- Personas conectadas por vínculos sociales.
- Ciudades conectadas por rutas o vuelos.
- Servidores conectados en una red.
- Productos o jugadores vinculados por recomendaciones.

Una vez modelado como grafo, podemos:

- Calcular medidas de importancia o influencia.
- Detectar comunidades o cuellos de botella.
- Visualizar la estructura completa del sistema.

¿Qué es un grafo?

Un **grafo** es una estructura matemática que representa relaciones entre elementos. Se define como $G = (V, E)$, donde:

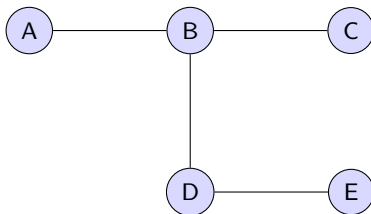
- V es el conjunto de **vértices o nodos** (entidades).
- E es el conjunto de **aristas o enlaces** (conexiones entre nodos).

Tipos básicos de grafos:

- **No dirigidos**: las conexiones son simétricas (amistad, colaboración).
- **Dirigidos**: las conexiones tienen dirección (seguir, recomendar).
- **Ponderados**: las aristas tienen un valor asociado (distancia, frecuencia, flujo).

Los ejercicios de esta primera parte trabajarán con **grafos no dirigidos y sin pesos**.

Definiciones



- **Grado de un nodo:** número de aristas que lo conectan con otros nodos. (*B* tiene grado 3.)
- **Nodos adyacentes:** dos nodos que están conectados por una arista.
- **Camino:** secuencia de nodos donde cada par consecutivo está conectado. (Camino: *A-B-D-E*.)
- **Componente conexa:** conjunto de nodos que están conectados entre sí.
- **Subgrafo:** grafo formado por un subconjunto de nodos y aristas de otro grafo. ($\{A, B, C\}$ forman un subgrafo conexo).

Creando grafos en Python con NetworkX

NetworkX es una librería de Python para crear, analizar y visualizar grafos.

Ejemplo básico:

```
import networkx as nx

# Crear un grafo vacío
G = nx.Graph()

# Agregar nodos
G.add_node("A")
G.add_nodes_from(["B", "C", "D", "E"])

# Agregar aristas (conexiones)
G.add_edge("A", "B")
G.add_edges_from([("B", "C"), ("B", "D"), ("D", "E")])
```

El grafo G ahora contiene:

- 5 nodos: A, B, C, D, E
- 4 aristas: A-B, B-C, B-D, D-E

Consultando información del grafo

Una vez creado el grafo, podemos inspeccionar su estructura:

```
# Listar nodos y aristas
print(G.nodes())
print(G.edges())

# Cantidad de nodos y aristas
print(G.number_of_nodes())
print(G.number_of_edges())

# Vecinos (nodos adyacentes)
print(list(G.neighbors("B")))
```

Salida esperada:

```
['A', 'B', 'C', 'D', 'E']
[('A', 'B'), ('B', 'C'), ('B', 'D'), ('D', 'E')]
5
4
['A', 'C', 'D']
```

Iteración sobre los nodos de un grafo

Una vez creado el grafo, podemos inspeccionar su estructura:

```
for nodo in G:  
    print(nodo)
```

Salida esperada:

A
B
C
D
E

Grado de los nodos en un grafo

El **grado** de un nodo indica cuántas aristas lo conectan con otros nodos.

Ejemplo: calcular el grado de cada nodo

```
# Recorrer todos los nodos y mostrar su grado
for nodo in G:
    print(nodo, "-> grado:", G.degree(nodo))
```

Salida esperada:

```
A -> grado: 1
B -> grado: 3
C -> grado: 1
D -> grado: 2
E -> grado: 1
```

El grado indica la **conectividad local** de cada nodo.

Conectividad del grafo

Un grafo es **conexo** si existe un camino entre cualquier par de nodos. NetworkX ofrece funciones para verificarlo fácilmente.

Ejemplo:

```
import networkx as nx

if nx.is_connected(G):
    print("La red es conexa.")
else:
    print("La red NO es conexa.")
```

Idea:

- Si el grafo es conexo, todos los nodos están en una misma “comunidad”.
- Si no lo es, hay grupos separados sin conexión entre sí.

Construir un grafo desde un archivo CSV

Podemos crear un grafo directamente leyendo un archivo CSV sin encabezado, donde cada fila contiene los dos nodos conectados.

Ejemplo de archivo colaboraciones.csv:

```
Ana,Bruno
Ana,Carla
Bruno,Diego
...
```

Construcción del grafo:

```
import networkx as nx

# Crear grafo no dirigido leyendo desde el CSV
G = nx.read_edgelist("colaboraciones.csv", delimiter=",")

print(G.nodes())
```

Nota: el archivo no debe tener encabezado, solo las parejas de nodos separadas por comas.

Visualizando un grafo

NetworkX permite dibujar grafos de manera sencilla con matplotlib.

```
import matplotlib.pyplot as plt

# Dibujo básico
nx.draw(G, with_labels=True)

plt.show()
```

- El parámetro `with_labels=True` muestra los nombres de los nodos.
- Podemos personalizar colores y tamaños:

```
nx.draw(G, with_labels=True,
        node_color="lightblue",
        node_size=800,
        font_size=10)
```

Ejercicio: red de comercio entre países

Ejercicio 1. Se dispone de información sobre vínculos comerciales entre países. Cada nodo representa un **país**, y una arista (u, v) indica que existe una relación comercial (exportaciones o importaciones) entre ambos.

El archivo `comercio.csv` (sin encabezado) tiene el siguiente formato:

```
Argentina,Brasil  
Brasil,Chile  
Chile,Peru  
Argentina,Uruguay  
Peru,Mexico  
...
```

Se pide:

- (a) Crear un grafo no dirigido G a partir del archivo `comercio.csv`.
- (b) Dibujar el grafo mostrando los nombres de los países.
- (c) Listar la cantidad de socios comerciales de cada país.
- (d) Determinar cuál es el país con más socios comerciales.
- (e) Verificar si toda la red está conectada o si existen grupos de países aislados.

Grafos ponderados

Hasta ahora trabajamos con grafos donde las aristas solo indicaban si existía o no una conexión.

En muchos problemas, las conexiones tienen una **intensidad**, **distancia** o **costo** asociado. En esos casos usamos un **grafo ponderado**.

Ejemplos típicos:

- Distancia entre ciudades.
- Tiempo de viaje entre estaciones.
- Costo de transporte o de comunicación.

Cada arista tiene un peso numérico (atributo `weight`).

Crear un grafo ponderado desde un CSV

Podemos leer un archivo CSV con tres columnas: Origen, Destino, Distancia (sin encabezado).

Ejemplo:

```
# rutas.csv
A,B,8
A,C,12
B,D,10
C,D,6
C,E,14
D,E,5
```

En Python:

```
import networkx as nx

G = nx.read_edgelist("rutas.csv",
                    delimiter="," ,
                    data=[("distancia", float)]) # lee la distancia
```

Acceder a información del grafo

En `networkx`, los grafos pueden manipularse como **diccionarios anidados**. Esto permite acceder fácilmente a la información sobre nodos, aristas y atributos.

- `G[nodo]` devuelve los **vecinos** del nodo.
- `G[nodo1][nodo2]` devuelve los **atributos** de la arista entre `nodo1` y `nodo2`.
- `G[nodo1][nodo2][atributo]` devuelve el **valor del atributo** (si existe).

Ejemplo:

```
G = nx.read_edgelist("rutas.csv", delimiter=",", data=[("distancia", int)])
print(G["BuenosAires"])           # vecinos de BuenosAires
print(G["BuenosAires"]["Cordoba"]) # atributos de la arista
print(G["BuenosAires"]["Cordoba"]["distancia"]) # distancia de la ruta
```

Salida esperada:

```
{'Rosario': {'distancia': 3}, 'Cordoba': {'distancia': 8}}
{'distancia': 8}
8
```

Visualización de un grafo ponderado

Podemos dibujar los nodos y también mostrar los pesos de las aristas.

```
import matplotlib.pyplot as plt

pos = nx.spring_layout(G, seed=42)           # posición automática
nx.draw(G, pos,
        with_labels=True,
        node_color="lightgreen",
        node_size=800,
        font_size=9)

# mostrar los pesos
labels = nx.get_edge_attributes(G, "distancia")
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

plt.show()
```

- `nx.spring_layout(G)` calcula posiciones equilibradas para los nodos, de modo que el grafo se visualice de forma clara.
- `nx.get_edge_attributes(G, "distancia")` devuelve un diccionario con los pesos asociados a cada arista.
- El parámetro opcional `edge_labels` asocia cada arista con su peso.

Ejercicio: red de alianzas entre empresas tecnológicas

Ejercicio 2. Se dispone de información sobre **alianzas y colaboraciones** entre empresas del sector tecnológico. Cada nodo representa una **empresa**, y una arista ponderada (u, v, w) indica que las empresas u y v colaboraron en w proyectos conjuntos durante el último año.

El archivo `alianzas.csv` tiene el siguiente formato:

```
AlphaTech,BetaSoft,3
AlphaTech,DataCorp,1
BetaSoft,InnoWare,4
DataCorp,InnoWare,2
InnoWare,Nexa,5
BetaSoft,Nexa,1
```

Se pide:

- Crear un grafo ponderado G a partir del archivo `alianzas.csv`.
- Dibujar la red de alianzas mostrando la cantidad de proyectos conjuntos entre dos empresas en las aristas.
- Calcular la cantidad de alianzas de cada empresa.
- Calcular la cantidad de colaboraciones de cada empresa.

Repaso de la clase

Grafos: **estructuras que representan relaciones** entre entidades (nodos y aristas).
Permiten modelar y analizar redes de transporte, colaboración, comercio, etc.

Vimos:

- Cómo crear grafos (dirigidos y no dirigidos) a partir de archivos CSV
- Cómo visualizar grafos y mostrar etiquetas de nodos y aristas
- Cómo obtener el grado y acceder a atributos de los grafos

Con lo visto, ya pueden completar hasta el **Ejercicio 2 de la Sección 4** de la guía de ejercicios.